2026 年度 大宮祭 研究発表

形式的冪級数と線形漸化式の高速計算

形式的冪級数 線形漸化式 高速計算 998244353 大好き

古寺 爽楽 数理科学研究会

2026-05-16

1. はじめに 2
2. 形式的冪級数 5
3. 線形漸化式 14
4. フィボナッチ数列の高速計算 24
5. 一般の線形漸化式への拡張 45
6. さいごに 51

2026 年度 大宮祭 研究発表

1. はじめに

競技プログラミングでは，形式的冪級数に関する問題が出題されることがある．
今回は，形式的冪級数の定義を数学的に確認し，それを用いた線形漸化式の高速な解法である Bostan–Mori 法を紹介する．

今回は以下の問題について考える．

問題 1

フィボナッチ数列の n 番目の項 F_n を，998244353 で割った余りを求めよ．
$(1 \leq n \leq 10^{18})$

2026 年度 大宮祭 研究発表

2. 形式的冪級数

まず形式的冪級数を定義する．

定義 1 (形式的冪級数)

可換環 R と形式的な文字 x をとる． R の元の列 $(a_0, a_1, a_2, \ldots)$ に対して，形式的な無限和

$$\sum_{n=0}^{\infty} a_n x^n$$

を考える．このような形式的な無限和全体の集合を $R[[x]]$ と書く．すなわち，

$$R[[x]] = \left\{ \sum_{n=0}^{\infty} a_n x^n \mid a_n \in R \right\}$$

である． $R[[x]]$ の元を形式的冪級数と呼ぶ．

形式的冪級数の例を挙げる.

例

数列 $a_n = n$ すなわち， $a_0 = 0, a_1 = 1, a_2 = 2, \cdots$ を考える.
この数列に対して, 形式的な文字 x を用いて, 以下のような形式的冪級数を考える.

$$\sum_{n=0}^{\infty} a_n x^n = \sum_{n=0}^{\infty} n x^n = 0 + 1x + 2x^2 + 3x^3 + \cdots$$

すなわち，数列の添え字 k の項 a_k は，形式的冪級数の x^k の係数になっている.

形式的冪級数の係数を取り出す記法を導入しておく．

定義 2

形式的冪級数

$$A(x) = \sum_{n=0}^{\infty} a_n x^n$$

に対して， x^k の係数 a_k を取り出す操作を以下のように書く．

$$[x^k] A(x) = a_k$$

この記号は以下のように扱う.

例

形式的冪級数

$$A(x) = \sum_{n=0}^{\infty} nx^n = 1 \cdot x + 2 \cdot x^2 + 3 \cdot x^3 + \cdots$$

に対して，x^2 の係数を取り出すことを考える.
このとき，x^2 の係数は 2 であるので，以下のように表される.

$$[x^2]A(x) = 2$$

また数列の母函数というものを導入する.

定義 3 (母函数)

数列 $(a_n)_{n=0}^{\infty}$ に対して，形式的な文字 x を用いて，以下のような形式的冪級数を考える.

$$\sum_{n=0}^{\infty} a_n x^n = a_0 + a_1 x + a_2 x^2 + a_3 x^3 + \cdots$$

このような形式的冪級数を数列 $(a_n)_{n=0}^{\infty}$ の母函数という.

すべての項が 1 である数列 $a_n = 1$ の母函数を $G(x)$ とすると，

$$G(x) = \sum_{n=0}^{\infty} x^n = 1 + x + x^2 + x^3 + \cdots$$

この母函数は，以下のような形で書くことがある．

$$G(x) = \frac{1}{1-x}$$

> **注意**
>
> 解析的な函数を考える場合には，無限級数の収束範囲を考える必要がある．
> いま x はあくまで形式的な文字であるので，収束などは考えない．

定義 4 (形式的冪級数の加法と乗法)

形式的冪級数の二つの元 A, B に対して以下で加法と乗法を定義する.

$$A = \sum_{n=0}^{\infty} a_n x^n, \quad B = \sum_{n=0}^{\infty} b_n x^n$$

- 加法

$$A + B = \sum_{n=0}^{\infty} (a_n + b_n) x^n$$

- 乗法

$$AB = \sum_{n=0}^{\infty} \left(\sum_{k=0}^{n} a_k b_{n-k} \right) x^n$$

例えば，$A(x) = x + 2x^2 + 3x^3, B(x) = 1 + x^2 + x^4$ の和と積に関しては，

$$A(x) + B(x) = (0+1) + (1+0)x + (2+1)x^2 + (3+0)x^3 + (0+1)x^4$$

$$= 1 + x + 3x^2 + 3x^3 + x^4$$

$$A(x)B(x) = (x + 2x^2 + 3x^3)(1 + x^2 + x^4)$$

$$= (x + 2x^2 + 3x^3) + (x^3 + 2x^4 + 3x^5) + (x^5 + 2x^6 + 3x^7)$$

$$= x + 2x^2 + 4x^3 + 2x^4 + 4x^5 + 2x^6 + 3x^7$$

> **注意**
>
> 形式的冪級数は可換環という代数的構造を持つ．
> また和は $O(n)$ ，積は $O(n^2)$ で計算できる．

2026 年度 大宮祭 研究発表

3. 線形漸化式

線形漸化式について定義する.

定義 5 (線形漸化式)

数列 $(a_n)_{n=0}^{\infty}$ に対して，正の整数 k と定数 $c_1, c_2, ..., c_k$ が存在し，
すべての $n \geq k$ に対して

$$a_n = c_1 a_{n-1} + c_2 a_{n-2} + \cdots + c_k a_{n-k}$$

を満たすとき，数列 $(a_n)_{n=0}^{\infty}$ は k 項線形漸化式を満たすという.

線形漸化式とは数列の各項が前の k 項の線形結合で表されるような関係式である.

例えば線形漸化式の例として以下のものがある．

例 (フィボナッチ数列)

$F_0 = 0, F_1 = 1$ であり，$n \geq 2$ に対して $F_n = F_{n-1} + F_{n-2}$ を満たす数列である．

これは，前の 2 項の線形結合で表されるような線形漸化式である．

例 (トリボナッチ数列)

$T_0 = 0, T_1 = 0, T_2 = 1$ であり，$n \geq 3$ に対して $T_n = T_{n-1} + T_{n-2} + T_{n-3}$ を満たす数列である．

これは，前の 3 項の線形結合で表されるような線形漸化式である．

母函数の中で特に有理型母函数と呼ばれるものがある．

定義 6 (有理型母函数)

係数環を体とする．数列 $(a_n)_{n=0}^{\infty}$ の母函数が多項式 $P(x), Q(x)$ を用いて以下のように表されるとする．

$$F(x) = \sum_{n=0}^{\infty} a_n x^n = \frac{P(x)}{Q(x)}, \quad (Q(0) \neq 0)$$

このとき，数列 $(a_n)_{n=0}^{\infty}$ の母函数 $F(x)$ は有理型母函数であるという．

有理型母函数の例として以下のものがある.

例 (有理型母函数の例)

有理型母函数の例としては以下のものがある.

$$F(x) = \frac{x+1}{x^2 - x + 1}, \quad G(x) = \frac{1}{1 - x - x^2}$$

命題 1

数列 $(a_n)_{n=0}^{\infty}$ が，正の整数 k と定数 $c_1, c_2, \ldots, c_k$ に対して

$$a_n = c_1 a_{n-1} + c_2 a_{n-2} + \cdots + c_k a_{n-k}$$

をすべての $n \geq k$ で満たすとする．このとき，その母函数は

$$F(x) = \frac{P(x)}{1 - c_1 x - c_2 x^2 - \cdots - c_k x^k}$$

という有理型母函数で表される．ここで，$P(x)$ は初期値によって決まる多項式である．

注意

すなわち，線形漸化式を満たす数列の母函数は有理型母函数である．

フィボナッチ数列は線形漸化式なので，その母函数は有理型母函数で表される．

命題 2

フィボナッチ数列 $(F_n)_{n=0}^{\infty}$ の母函数は以下で表される．

$$F(x) = \sum_{n=0}^{\infty} F_n x^n = \frac{x}{1 - x - x^2}$$

証明: フィボナッチ数列 $(F_n)_{n=0}^{\infty}$ の母函数を $F(x)$ とおく．すなわち，

$$F(x) = x + x^2 + 2x^3 + 3x^4 + 5x^5 + \cdots$$

ここで， $F(x)$ を x 倍， x^2 倍すると

$$F(x) = x + x^2 + 2x^3 + 3x^4 + 5x^5 + \cdots$$

$$xF(x) = x^2 + x^3 + 2x^4 + 3x^5 + 5x^6 + \cdots$$

$$x^2F(x) = x^3 + x^4 + 2x^5 + 3x^6 + 5x^7 + \cdots$$

ここで，$F(x) - xF(x) - x^2F(x)$ を考えると，$F_{n+2} = F_{n+1} + F_n$ より，

$$
\begin{aligned}
F(x) - xF(x) - x^2F(x) &= (x + x^2 + 2x^3 + 3x^4 + 5x^5 + \cdots) \\
&\quad -(x^2 + x^3 + 2x^4 + 3x^5 + 5x^6 + \cdots) \\
&\quad -(x^3 + x^4 + 2x^5 + 3x^6 + 5x^7 + \cdots) \\
&= x + (x^2 - x^2) + (2x^3 - x^3 - x^3) + (3x^4 - 2x^4 - x^4) + \cdots \\
&= x
\end{aligned}
$$

したがって以下を得る．

$$F(x) - xF(x) - x^2F(x) = x$$

$$F(x)(1 - x - x^2) = x$$

$$F(x) = \frac{x}{1 - x - x^2}$$

すなわち，フィボナッチ数列の母函数 $F(x)$ は，

$$F(x) = \frac{x}{1 - x - x^2}$$

□

2026 年度 大宮祭 研究発表

4. フィボナッチ数列の高速計算

今回考える問題をもう一度確認する．

問題 2

フィボナッチ数列の n 番目の項 F_n を，998244353 で割った余りを求めよ．
$(1 \leq n \leq 10^{18})$

注意

競技プログラミングでは，一部の言語では数列の値が非常に大きくなるとオーバーフローをしてしまうことが多いため，素数 998244353 で割った余りを求める形で出題されることが多い．

単純な方法として F_n を求めるために $F_0, F_1, ..., F_{n-1}$ を順に計算する方法を考える．
例えば， F_5 は以下のように計算する．

$$F_0 = 0$$

$$F_1 = 1$$

$$F_2 = F_1 + F_0 = 1 + 0 = 1$$

$$F_3 = F_2 + F_1 = 1 + 1 = 2$$

$$F_4 = F_3 + F_2 = 2 + 1 = 3$$

$$F_5 = F_4 + F_3 = 3 + 2 = 5$$

いま F_5 はとても小さい添え字なのですぐ計算が終わるが，$F_{10^{18}}$ ではどうだろうか．
$F_{10^{18}}$ を求めるために，F_0 から $F_{10^{18}-1}$ までを計算しなくてはならない．
このやり方は時間計算量が $O(n)$ であり，n が大きい場合では現実的でない．

> **注意**
>
> 競技プログラミングの問題では実行時間制限が 2 秒程度であることが多い．
> 一般に，10^8 回程度の計算であれば，実行時間制限に収まることが多い．

フィボナッチ数列は行列累乗を用いて高速に計算することができる．
まず以下のようにフィボナッチ数列を行列の形で表すことができる．

$$\begin{pmatrix} F_{n+1} \\ F_n \end{pmatrix} = \begin{pmatrix} F_n + F_{n-1} \\ F_n \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} F_n \\ F_{n-1} \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}^2 \begin{pmatrix} F_{n-1} \\ F_{n-2} \end{pmatrix}$$

したがって，

$$\begin{pmatrix} F_{n+1} \\ F_n \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}^n \begin{pmatrix} F_1 \\ F_0 \end{pmatrix}$$

よって F_n を求めるためには，以下の行列を計算すればいい．

$$\begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}^n$$

例えば，F_5 を計算するためには以下のように計算すればよい．

$$\begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}^5 = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}\begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}\begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}\begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}\begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix} = \begin{pmatrix} 8 & 5 \\ 5 & 3 \end{pmatrix}$$

よって，

$$\begin{pmatrix} F_6 \\ F_5 \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}^5 \begin{pmatrix} F_1 \\ F_0 \end{pmatrix} = \begin{pmatrix} 8 & 5 \\ 5 & 3 \end{pmatrix}\begin{pmatrix} 1 \\ 0 \end{pmatrix} = \begin{pmatrix} 8 \\ 5 \end{pmatrix}$$

このことから，$F_5 = 5$ であることがわかる．

> **注意**
>
> 行列の累乗に関しては繰り返し二乗法などを用いて計算すれば $O(\log n)$ で計算することができる．

Bostan–Mori 法は，有理型母函数

$$F(x) = \frac{P(x)}{Q(x)}$$

の x^n の係数を高速に求める方法である．すなわち，以下を高速に計算できる．

$$[x^n]\frac{P(x)}{Q(x)}$$

線形漸化式を満たす数列の母函数は有理型母函数で表される．
この方法を用いると線形漸化式を満たす数列の n 番目の項を高速に求められる．

> **注意**
>
> 大まかな方針は，係数を取り出す添え字 n を毎回半分にし，同じ形の問題に戻すことである．

いま，以下の有理型母函数の x^n の係数を求めることを考える．

$$F(x) = \frac{P(x)}{Q(x)}$$

ここで，分母分子に $Q(-x)$ をかけると，

$$\frac{P(x)}{Q(x)} = \frac{P(x)Q(-x)}{Q(x)Q(-x)}$$

となる．
ここで $Q(x)Q(-x)$ は偶数次数の項だけを持つ多項式になる．
したがって，ある多項式 $T_e(x)$ を用いて

$$Q(x)Q(-x) = T_e(x^2)$$

と表せる．

次に分子を考える．

$$S(x) = P(x)Q(-x), \quad T(x) = Q(x)Q(-x)$$

とおく．
分母 $T(x)$ は偶数次数の項だけを持つので，

$$T(x) = T_e(x^2)$$

と書ける．また，分子 $S(x)$ を偶数次数部分と奇数次数部分に分けると，

$$S(x) = S_e(x^2) + xS_o(x^2)$$

と書ける．

したがって，

$$\frac{S(x)}{T(x)} = \frac{S_e(x^2)}{T_e(x^2)} + x\frac{S_o(x^2)}{T_e(x^2)}$$

である．
このとき，求めたい添え字 n が偶数なら，

$$[x^n]\frac{S(x)}{T(x)} = \left[x^{\frac{n}{2}}\right]\frac{S_e(x)}{T_e(x)}$$

であり，n が奇数なら，

$$[x^n]\frac{S(x)}{T(x)} = \left[x^{\frac{n-1}{2}}\right]\frac{S_o(x)}{T_e(x)}$$

である．

n の偶奇に応じて分子の偶数次数部分または奇数次数部分を取り出し，次数を半分にすればよい．最後に $n = 0$ となったら，答えは $\frac{P(0)}{Q(0)}$ である．

```
while n > 0:
    S = P(x) Q(-x)
    T = Q(x) Q(-x)

    if n が偶数:
        P = S の偶数次数部分を取り出し，次数を半分にする
    else:
        P = S の奇数次数部分を取り出し，次数を半分にする

    Q = T の偶数次数部分を取り出し，次数を半分にする
    n = floor(n / 2)
```

フィボナッチ数列でこの操作を見てみる．
フィボナッチ数列の母函数は

$$F(x) = \frac{x}{1 - x - x^2}$$

であった．
したがって，

$$P(x) = x, \quad Q(x) = 1 - x - x^2$$

から始めればよい．
例えば F_{10} を求めることを考えると，

$$F_{10} = [x^{10}] \frac{x}{1 - x - x^2}$$

である．

まず，

$$Q(-x) = 1 + x - x^2$$

であるので，

$$Q(x)Q(-x) = (1 - x - x^2)(1 + x - x^2) = 1 - 3x^2 + x^4$$

$$P(x)Q(-x) = x(1 + x - x^2) = x + x^2 - x^3$$

となる．
いま $n = 10$ は偶数なので，分子の偶数次数部分だけを残す．
分子は x^2 を取り出して次数を半分にするので $P(x) = x$ となり，分母は $1 - 3x^2 + x^4$ の次数を半分にするので $Q(x) = 1 - 3x + x^2$ となる．

したがって，

$$[x^{10}]\frac{x}{1-x-x^2} = [x^5]\frac{x}{1-3x+x^2}$$

と変換できる．
この時点で，添え字が 10 から 5 に半分になっている．
同じ操作を繰り返すと，以下のようになる．

n	$P(x)$	$Q(x)$
10	x	$1-x-x^2$
5	x	$1-3x+x^2$
2	$1+x$	x^2-7x+1
1	$1+8x$	$x^2-47x+1$
0	$55+8x$	$x^2-2207x+1$

最後の行で $n = 0$ となっている．
この例では $Q(0) = 1$ であるため，求めたい値は $P(0)$ になる．
したがって，

$$F_{10} = 55$$

である．

> **注意**
>
> 実際には F_{10} 程度なら普通に計算できるが，この方法では n が大きくても，添え字を半分にしながら進められる．

ここまでで，フィボナッチ数列の F_n を求める方法として，単純な方法，行列累乗，Bostan–Mori 法を見た．それぞれの計算量は以下のようになる．

方法	時間計算量
単純な方法	$O(n)$
行列累乗	$O(\log n)$
Bostan–Mori 法	$O(\log n)$

> **注意**
>
> フィボナッチ数列は 2 項間漸化式なので，行列累乗や Bostan–Mori 法において行列や多項式のサイズが小さいため，両者の計算量は同じオーダーになる．

ここで，実際にフィボナッチ数列の $F_{10^{18}} \bmod 998244353$ を求めてみる．
(~~1 分間時間を取るので計算してみましょう！！~~)

0–20 回

回	n	$P(x)$	$Q(x)$
0	1000000000000000000	x	$1+998244352x+998244352x^2$
1	500000000000000000	x	$1+998244350x+x^2$
2	250000000000000000	$3x$	$1+998244346x+x^2$
3	125000000000000000	$21x$	$1+998244306x+x^2$
4	62500000000000000	$987x$	$1+998242146x+x^2$
5	31250000000000000	$2178309x$	$1+993373506x+x^2$
6	15625000000000000	$868874039x$	$1+123040344x+x^2$
7	7812500000000000	$68921677x$	$1+392489522x+x^2$
8	3906250000000000	$341969642x$	$1+509822633x+x^2$
9	1953125000000000	$104186878x$	$1+105786151x+x^2$
10	976562500000000	$245887711x$	$1+765521577x+x^2$
11	488281250000000	$942839953x$	$1+650055658x+x^2$
12	244140625000000	$188323242x$	$1+823264894x+x^2$
13	122070312500000	$57949216x$	$1+320203301x+x^2$
14	61035156250000	$490320229x$	$1+686134135x+x^2$
15	30517578125000	$688849975x$	$1+566512014x+x^2$
16	15258789062500	$539562357x$	$1+949447118x+x^2$
17	7629394531250	$123638574x$	$1+9738151x+x^2$
18	3814697265625	$360202863x$	$1+630391848x+x^2$
19	1907348632812	$360202863+360202863x$	$1+557392827x+x^2$
20	953674316406	$360202863+189826315x$	$1+810180613x+x^2$

21–40 回

回	n	$P(x)$	$Q(x)$
21	476837158203	$360202863 + 964845067x$	$1 + 818519824x + x^2$
22	238418579101	$581893056 + 964845067x$	$1 + 948705909x + x^2$
23	119209289550	$955132709 + 964845067x$	$1 + 529120888x + x^2$
24	59604644775	$955132709 + 639535304x$	$1 + 831596476x + x^2$
25	29802322387	$127687463 + 639535304x$	$1 + 363331247x + x^2$
26	14901161193	$773827557 + 639535304x$	$1 + 95428038x + x^2$
27	7450580596	$631203945 + 639535304x$	$1 + 626310589x + x^2$
28	3725290298	$631203945 + 493052959x$	$1 + 113469013x + x^2$
29	1862645149	$631203945 + 435485809x$	$1 + 996079119x + x^2$
30	931322574	$348486815 + 435485809x$	$1 + 515451287x + x^2$
31	465661287	$348486815 + 420785603x$	$1 + 450725357x + x^2$
32	232830643	$614442124 + 420785603x$	$1 + 710882826x + x^2$
33	116415321	$985885918 + 420785603x$	$1 + 383283207x + x^2$
34	58207660	$760978465 + 420785603x$	$1 + 519007411x + x^2$
35	29103830	$760978465 + 798299746x$	$1 + 255062754x + x^2$
36	14551915	$760978465 + 565681548x$	$1 + 501637217x + x^2$
37	7275957	$379200613 + 565681548x$	$1 + 695309411x + x^2$
38	3637978	$607806103 + 565681548x$	$1 + 569723872x + x^2$
39	1818989	$607806103 + 724225370x$	$1 + 254048568x + x^2$
40	909494	$415770386 + 724225370x$	$1 + 171216683x + x^2$

41–60 回

回	n	$P(x)$	$Q(x)$
41	454747	$415770386 + 654429120x$	$1 + 884410496x + x^2$
42	227373	$30172864 + 654429120x$	$1 + 57402314x + x^2$
43	113686	$326562238 + 654429120x$	$1 + 270959219x + x^2$
44	56843	$326562238 + 902261515x$	$1 + 272974160x + x^2$
45	28421	$316095778 + 902261515x$	$1 + 44648634x + x^2$
46	14210	$748235561 + 902261515x$	$1 + 442585517x + x^2$
47	7105	$748235561 + 858018729x$	$1 + 772357798x + x^2$
48	3552	$988745460 + 858018729x$	$1 + 295541652x + x^2$
49	1776	$988745460 + 944144461x$	$1 + 480416103x + x^2$
50	888	$988745460 + 599858326x$	$1 + 651563837x + x^2$
51	444	$988745460 + 29257856x$	$1 + 107687320x + x^2$
52	222	$988745460 + 642116613x$	$1 + 839022717x + x^2$
53	111	$988745460 + 642358171x$	$1 + 14616454x + x^2$
54	55	$957350941 + 642358171x$	$1 + 534161887x + x^2$
55	27	$40478255 + 642358171x$	$1 + 595410453x + x^2$
56	13	$920452748 + 642358171x$	$1 + 68864152x + x^2$
57	6	$189136456 + 642358171x$	$1 + 171614934x + x^2$
58	3	$189136456 + 222094397x$	$1 + 725001498x + x^2$
59	1	$180138096 + 222094397x$	$1 + 141558681x + x^2$
60	0	$23849548 + 222094397x$	$1 + 793074953x + x^2$

最後の行で $P(x) = 23849548 + 222094397x, Q(x) = 1 + 793074953x + x^2$ となり，

$$F_{10^{18}} = [x^0]\frac{23849548 + 222094397x}{1 + 793074953x + x^2}$$

と帰着できる．したがって

$$F_{10^{18}} \equiv 23849548 \bmod 998244353$$

このように，非常に大きい添え字でも，60 回程度の更新で求めることができる．

2026 年度 大宮祭 研究発表

5. 一般の線形漸化式への拡張

次に，一般の k 項線形漸化式を考える．

問題 3

数列 $(a_n)_{n=0}^{\infty}$ が，次の k 項線形漸化式

$$a_n = c_1 a_{n-1} + c_2 a_{n-2} + \cdots + c_k a_{n-k} \quad (n \geq k)$$

を満たすとする．
さらに，初期値

$$a_0 = d_0, \quad a_1 = d_1, \quad \ldots, \quad a_{k-1} = d_{k-1}$$

が与えられているとする．
このとき，第 n 項 a_n を 998244353 で割った余りを求めよ．$(1 \leq n \leq 10^{18})$

フィボナッチ数列と同じように，この問題も行列累乗で考えることができる．
まず，以下の $k \times k$ 行列を考える．

$$C = \begin{pmatrix} c_1 & c_2 & \cdots & c_{k-1} & c_k \\ 1 & 0 & \cdots & 0 & 0 \\ 0 & 1 & \cdots & 0 & 0 \\ \vdots & \vdots & \vdots & \vdots & \vdots \\ 0 & 0 & \cdots & 1 & 0 \end{pmatrix}$$

このとき，

$$\begin{pmatrix} a_n \\ a_{n-1} \\ \vdots \\ a_{n-k+1} \end{pmatrix} = C \begin{pmatrix} a_{n-1} \\ a_{n-2} \\ \vdots \\ a_{n-k} \end{pmatrix}$$

と書ける．
この $k \times k$ 行列を繰り返し二乗法で累乗すれば，a_n を求められる．
ただし，行列積に $O(k^3)$ かかるため，全体では $O(k^3 \log n)$ になる．

一方，Bostan–Mori 法では母函数を使う．
線形漸化式を満たす数列の母函数は，

$$F(x) = \frac{P(x)}{1 - c_1 x - c_2 x^2 - \cdots - c_k x^k}$$

という形になる．
分母を

$$Q(x) = 1 - c_1 x - c_2 x^2 - \cdots - c_k x^k$$

とおく．また，初期値から

$$A(x) = a_0 + a_1 x + \cdots + a_{k-1} x^{k-1}$$

とおくと，分子は

$$P(x) \equiv Q(x) A(x) \bmod x^k$$

として求められる．

あとは，フィボナッチ数列の場合と同じように

$$[x^n]\frac{P(x)}{Q(x)}$$

を Bostan–Mori 法で計算すればよい．
このとき，添え字 n は毎回半分になるので，更新回数は $O(\log n)$ 回である．
多項式積を素朴に計算すると 1 回の更新に $O(k^2)$ かかるため，全体の計算量は

$$O(k^2 \log n)$$

より一般には，多項式積の計算量を $M(k)$ と書けば， $O(M(k)\log n)$ と表せる．

一般の k 項線形漸化式に対する計算量をまとめると以下のようになる．

方法	時間計算量
単純な方法	$O(kn)$
行列累乗	$O(k^3 \log n)$
Bostan–Mori 法	$O(M(k) \log n)$
Bostan–Mori 法，素朴な積	$O(k^2 \log n)$
Bostan–Mori 法，NTT	$O(k \log k \log n)$

特に k が大きい場合，行列累乗よりも Bostan–Mori 法の方が有利になることがある．

2026 年度 大宮祭 研究発表

6. さいごに

本発表では，形式的冪級数を数列を扱うための対象として導入した．
また，線形漸化式を満たす数列の母函数が有理型母函数で表されることを確認した．
さらに，Bostan–Mori 法により，

$$[x^n]\frac{P(x)}{Q(x)}$$

を添え字 n を半分にしながら高速に求められることを見た．
今回は実装までは扱わなかったが，競技プログラミングでは線形漸化式の第 n 項を高速に求める場面で利用できる．

> **注意**
>
> 多項式積に NTT(数論変換) を用いると，Bostan–Mori 法はさらに高速になる．
> また，同じ問題を解く方法として Kitamasa 法も知られている．